

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Tecnología de Objetos				
TÍTULO DE LA PRÁCTICA:	Laboratorio 02				
NÚMERO DE PRÁCTICA:	2	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VI
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	MÍNIMO DE ESTUDIANTES	-----	MÁXIMO DE ESTUDIANTES	2
FECHA INICIO:	09/09/2026	FECHA FIN:	12/09/2026	DURACIÓN:	2 horas
INTEGRANTE (s): Chipana Jeronimo German Arturo Baca Calsin Leonardo Juan José					
DOCENTE(s): Edith Giovanna Cano Mamani					



UNIVERSIDAD NACIONAL DE SAN AGUSTIN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLD-001

Página: 2

CONTENIDO DE LA GUÍA

I. SOLUCIÓN

Repositorio:

<https://github.com/ChipanaGerman/TecnologiaDeObjetos-Lab-B/tree/main/Lab2>

INTRODUCCIÓN

El paradigma de la Programación Orientada a Objetos (POO) constituye la base del desarrollo de software moderno al permitir modelar entidades del mundo real mediante abstracciones modulares y reutilizables. No obstante, el verdadero potencial de la POO no radica únicamente en la definición aislada de clases, sino en el establecimiento estructurado de las interacciones y relaciones semánticas entre ellas.

En este laboratorio se aborda la implementación práctica de las relaciones fundamentales en Java: herencia, agregación, composición y dependencia. A través de un caso de estudio académico compuesto por universidades, cursos, horarios, profesores, estudiantes y reportes, se aplican los principios de encapsulamiento mediante el estándar JavaBeans. Con ello, se consolida la comprensión del ciclo de vida de los objetos y el acoplamiento entre módulos dentro de una arquitectura orientada a objetos.

OBJETIVOS

Objetivo General

- Diseñar e implementar un sistema orientado a objetos en Java que represente adecuadamente los distintos tipos de relaciones estructurales y de comportamiento entre clases (herencia, agregación, composición y dependencia), garantizando la integridad de los datos mediante el encapsulamiento.

Objetivos Específicos

- Aplicar la **herencia** extendiendo la clase base Persona hacia clases especializadas (Profesor y Estudiante).
- Demostrar la **composición** entre las clases Curso y Horario, evidenciando el ciclo de vida fuertemente ligado entre ambas entidades.
- Implementar la **agregación** en la clase Universidad, gestionando una colección de instancias independientes de Curso.
- Ilustrar la relación de **dependencia** a través de la clase Reporte, la cual utiliza de forma transitoria los datos de un Estudiante sin mantener una referencia persistente.
- Aplicar la convención estándar **JavaBeans** (atributos privados, constructores sobrecargados, métodos accesorios/modificadores y sobrescritura del método toString()).

MARCO CONCEPTUAL

La Programación Orientada a Objetos se fundamenta en la cohesión interna de las entidades y el desacoplamiento entre ellas, empleando mecanismos definidos por el estándar UML (*Unified Modeling Language*) [1].

Encapsulamiento y Convención JavaBeans

El encapsulamiento es el mecanismo que restringe el acceso directo a los componentes internos de un objeto, protegiendo su estado mediante modificadores de visibilidad (*private*) y exponiendo interfaces controladas [2]. La convención **JavaBeans** formaliza esta técnica exigiendo constructores públicos, métodos de acceso (*getters*) y de modificación (*setters*), así como la implementación de métodos utilitarios como *toString()* para la serialización y representación textual del estado [3].

Herencia (Generalización / Especialización)

La herencia permite crear nuevas clases a partir de clases existentes, reutilizando y extendiendo atributos y métodos comunes [1]. Se clasifica como una relación de tipo «es un» (*is-a*). En Java se logra mediante la palabra clave *extends*, fomentando el polimorfismo y reduciendo la duplicación de código [3].

Composición

La composición es una forma fuerte de asociación («parte-todo») en la que la parte no tiene sentido de existencia sin el todo [4]. Si el objeto contenedor deja de existir, los objetos contenidos también son destruidos. En código, el objeto contenedor se encarga de crear, encapsular y controlar el ciclo de vida de sus partes [2].

Agregación

La agregación representa una forma débil de asociación («tiene un» / *has-a*) donde las partes pueden existir de manera independiente al contenedor [4]. Las referencias a los objetos agregados suelen inyectarse desde el exterior (vía constructor o métodos de asignación), lo que permite que el ciclo de vida de los componentes trascienda al del contenedor [1].

Dependencia

La dependencia es una relación de uso temporal («usa un» / *uses-a*), donde una clase requiere de otra para ejecutar una operación específica (típicamente recibida como parámetro de un método o instanciada localmente) [1], [4]. Si la clase proveedora sufre cambios estructurales, la clase dependiente puede verse afectada, por lo que debe mantenerse al mínimo para evitar un acoplamiento excesivo [2].

Problema Propuesto:

Java

```
import java.util.ArrayList;
import java.util.List;

// Clase base
class Persona {
    private String nombre;
    private int edad;

    public Persona() {}

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }

    public String toString() {
        return "Persona [nombre=" + nombre + ", edad=" + edad + "]";
    }
}

// Clases hijas
class Profesor extends Persona {
    private String especialidad;

    public Profesor() {
        super();
    }

    public Profesor(String nombre, int edad, String especialidad) {
        super(nombre, edad);
    }
}
```

```
        this.especialidad = especialidad;
    }

    public String getEspecialidad() {
        return especialidad;
    }

    public void setEspecialidad(String especialidad) {
        this.especialidad = especialidad;
    }

    public String toString() {
        return "Profesor [" + super.toString() + ", especialidad=" + especialidad + "]";
    }
}

class Estudiante extends Persona {
    private String codigoEstudiante;

    public Estudiante() {
        super();
    }

    public Estudiante(String nombre, int edad, String codigoEstudiante) {
        super(nombre, edad);
        this.codigoEstudiante = codigoEstudiante;
    }

    public String getCodigoEstudiante() {
        return codigoEstudiante;
    }

    public void setCodigoEstudiante(String codigoEstudiante) {
        this.codigoEstudiante = codigoEstudiante;
    }

    public String toString() {
        return "Estudiante [" + super.toString() + ", codigo=" + codigoEstudiante + "]";
    }
}

// Clases de cursos y horarios
class Horario {
    private String dias;
    private String horas;
```

```
public Horario() {}

public Horario(String dias, String horas) {
    this.dias = dias;
    this.horas = horas;
}

public String getDias() {
    return dias;
}

public void setDias(String dias) {
    this.dias = dias;
}

public String getHoras() {
    return horas;
}

public void setHoras(String horas) {
    this.horas = horas;
}

public String toString() {
    return "Horario [dias=" + dias + ", horas=" + horas + "]";
}
}

class Curso {
    private String nombre;
    // El horario se inicializa junto con el curso
    private Horario horario;

    public Curso(String nombre, String dias, String horas) {
        this.nombre = nombre;
        this.horario = new Horario(dias, horas);
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

```
public Horario getHorario() {
    return horario;
}

public void setHorario(Horario horario) {
    this.horario = horario;
}

public String toString() {
    return "Curso [nombre=" + nombre + ", " + horario.toString() + "]";
}
}

// Entidad principal que agrupa los cursos
class Universidad {
    private String nombre;
    private List<Curso> listaCursos;

    public Universidad() {
        this.listaCursos = new ArrayList<>();
    }

    public Universidad(String nombre) {
        this.nombre = nombre;
        this.listaCursos = new ArrayList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public List<Curso> getListaCursos() {
        return listaCursos;
    }

    // Método para ir agregando cursos a la lista
    public void agregarCurso(Curso curso) {
        this.listaCursos.add(curso);
    }

    public String toString() {
```

```
        return "Universidad [nombre=" + nombre + ", cantidadCursos=" + listaCursos.size() +
    "]" +
    ";
    }
}

// Clase para emitir reportes
class Reporte {
    public Reporte() {}

    // Imprime los datos del estudiante ingresado
    public void generarReporteEstudiante(Estudiante estudiante) {
        System.out.println("=====");
        System.out.println("        REPORTE ACADÉMICO DE ESTUDIANTE        ");
        System.out.println("=====");
        System.out.println("Nombre: " + estudiante.getNombre());
        System.out.println("Edad:   " + estudiante.getEdad() + " años");
        System.out.println("Código: " + estudiante.getCodigoEstudiante());
        System.out.println("Estado: ACTIVO REGULAR");
        System.out.println("=====\\n");
    }

    @Override
    public String toString() {
        return "Reporte [modulo=Generador de reportes temporales]";
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("        SISTEMA DE GESTIÓN UNIVERSITARIA        ");
        System.out.println("=====\\n");

        // Registro de profesores
        Profesor prof1 = new Profesor("Dr. Carlos Ruiz", 48, "Sistemas Distribuidos");
        Profesor prof2 = new Profesor("MSc. Ana Paredes", 39, "Arquitectura de Software");

        System.out.println("--- Profesores Registrados ---");
        System.out.println(prof1);
        System.out.println(prof2);
        System.out.println();

        // Registro de estudiantes
        Estudiante est1 = new Estudiante("Juan Perez", 20, "20220101");
        Estudiante est2 = new Estudiante("Maria Lopez", 22, "20210452");
        Estudiante est3 = new Estudiante("Luis Quispe", 21, "20221189");
```

```
System.out.println("--- Estudiantes Registrados ---");
System.out.println(est1);
System.out.println(est2);
System.out.println(est3);
System.out.println();

// Instanciando cursos
Curso curso1 = new Curso("Tecnologia de Objetos", "Lunes y Miercoles", "08:00 -
10:00");
Curso curso2 = new Curso("Bases de Datos Avanzadas", "Martes y Jueves", "14:00 -
16:00");

System.out.println("--- Cursos Creados (con Horario por Composición) ---");
System.out.println(curso1);
System.out.println(curso2);
System.out.println();

// Agregando cursos a la universidad
Universidad uni = new Universidad("Universidad Nacional Tecnológica");
uni.agregarCurso(curso1);
uni.agregarCurso(curso2);

System.out.println("--- Universidad y Cursos Agregados ---");
System.out.println(uni);
for (Curso c : uni.getListaCursos()) {
    System.out.println(" -> " + c);
}
System.out.println();

// Prueba del reporte
Reporte reporte = new Reporte();
reporte.generarReporteEstudiante(est1);
}
}
```

Explicación Técnica de la Implementación

A. Estructuración y Procesamiento de Datos

Para permitir que el programa maneje estados y colecciones robustas, se utilizaron herramientas de la biblioteca estándar de Java y convenciones de diseño:

- **import java.util.List y java.util.ArrayList:** Se importan las interfaces y clases necesarias para instanciar estructuras de datos dinámicas, permitiendo almacenar colecciones de tamaño variable (como los

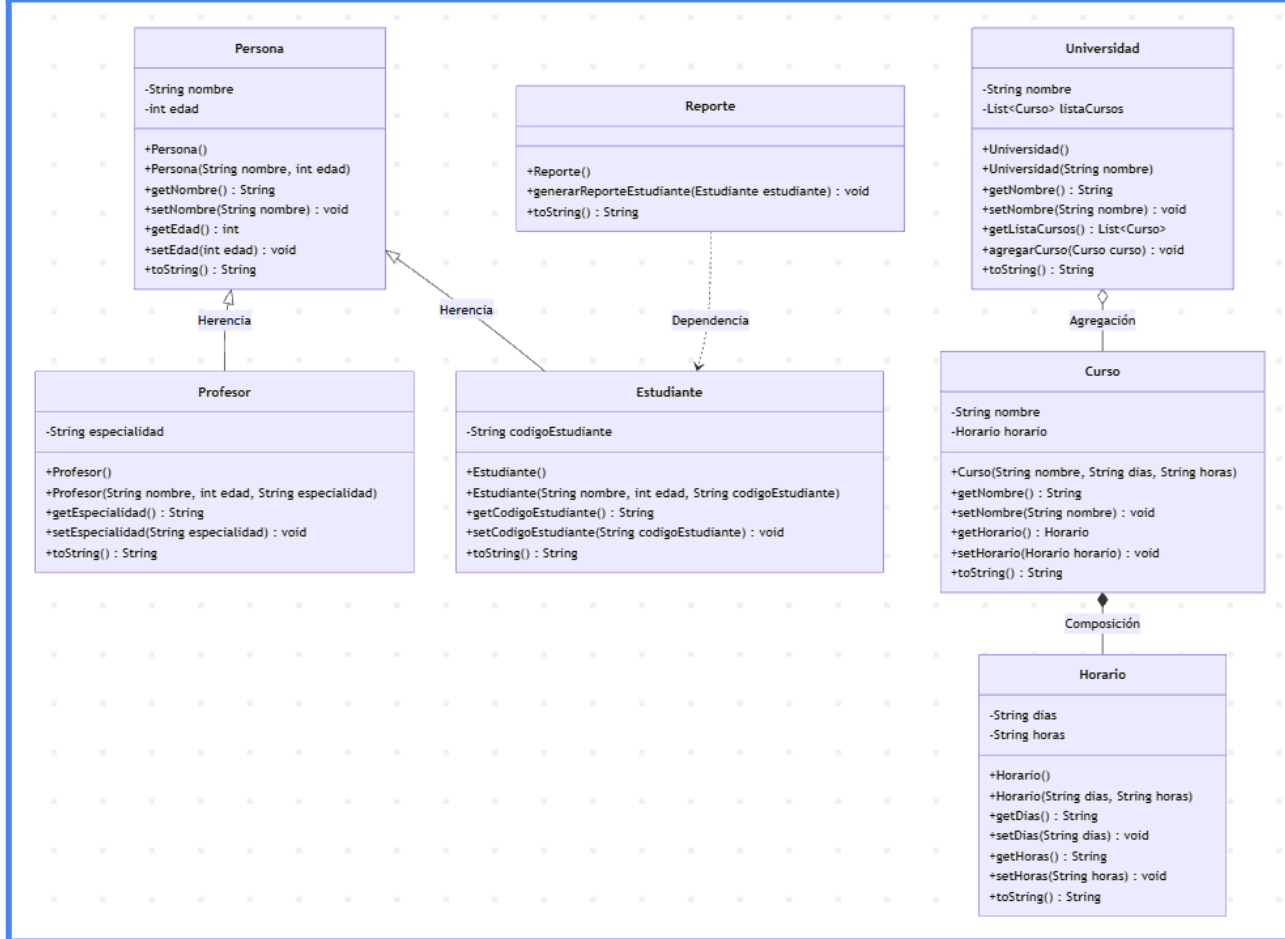
cursos dentro de la universidad).

- **Modificadores de acceso (private):** Se protege el estado interno de cada clase, ocultando los atributos (como nombre, edad o codigoEstudiante) para evitar modificaciones externas directas.
- **Gestión de instanciación (Operador new y Constructores):**
 - Se utiliza para reservar la memoria dinámicamente.
 - Los constructores sobrecargados capturan los valores de entrada y realizan el "binding" hacia los atributos propios de la clase utilizando la palabra reservada `this` (ej. `this.nombre = nombre`).

B. Análisis de la Lógica: Relaciones Estructurales

La solución utiliza una estrategia de diseño modular basada en el principio de que los objetos delegan responsabilidades y se comunican a través de cuatro mecanismos precisos:

1. **Herencia (extends y super()):** Se abstrae la base lógica en la clase `Persona`. Las clases derivadas (`Profesor` y `Estudiante`) se especializan añadiendo sus propios atributos. Al utilizar `super(nombre, edad)` dentro del constructor de la clase hija, el algoritmo delega la inicialización de los datos comunes directamente al constructor de la clase padre (la superclase). Esto permite reutilizar el código base y extender la funcionalidad.
2. **Composición (this.horario = new Horario(...)):** Se implementa una dependencia existencial estricta. Dentro de la firma del constructor de la clase `Curso`, se instancia directamente el objeto `Horario`. La lógica dicta que la parte (`Horario`) nace y muere con el todo (`Curso`), imposibilitando que exista un horario en el sistema de manera huérfana.
3. **Agregación (listaCursos.add(curso)):** La solución aborda una agrupación flexible para la clase `Universidad`. A diferencia de la composición, la universidad recibe los objetos `Curso` ya instanciados desde el exterior a través del paso de parámetros en el método `agregarCurso`. Esto significa que si la instancia de la universidad es eliminada, las referencias a los cursos continúan existiendo independientemente.
4. **Dependencia (generarReporteEstudiante(Estudiante estudiante)):** Se modela una relación de uso temporal. La clase `Reporte` no necesita poseer a un estudiante como atributo propio. Simplemente recibe al objeto `Estudiante` como parámetro en su método, extrayendo sus datos mediante los *getters* para imprimir la salida y descartando la interacción una vez finaliza el flujo de la función.



El diagrama UML modela la estructura estática del sistema y define los niveles de acoplamiento, sirviendo como mapa arquitectónico para la implementación en código.

- **Encapsulamiento y Visibilidad** El modelo aplica el principio de ocultación de la información. Todos los atributos llevan el símbolo - (privado), lo que protege los datos de modificaciones externas directas. En contraste, los constructores y métodos llevan el símbolo + (público), estableciendo los únicos canales seguros para leer o modificar el estado de cada objeto.
- **Herencia (Generalización)** Representada por las líneas con un triángulo hueco. Indica que Profesor y Estudiante son especializaciones de la superclase Persona. Al exponer, puedes destacar que esto permite heredar atributos base como nombre y edad, lo que elimina la redundancia de código y facilita agregar nuevos tipos de usuarios en el futuro sin reescribir la lógica central.
- **Composición (Ciclo de Vida Estricto)** Se grafica con el rombo sólido (negro) entre Curso y Horario. Modela una relación existencial fuerte: el horario nace y muere internamente con el curso. Si el objeto Curso es destruido por el recolector de basura, su Horario desaparece simultáneamente, ya que a nivel de negocio no tiene sentido que exista un horario huérfano.
- **Agregación (Agrupación Independiente)** Indicada por el rombo hueco (blanco) entre Universidad y Curso. Modela una colección donde las partes tienen autonomía. La universidad agrupa los cursos en una lista, pero si la instancia de la universidad es eliminada, los objetos Curso pueden seguir existiendo intactos en la memoria del sistema.

- **Dependencia (Interacción Temporal)** Mostrada con la flecha discontinua desde Reporte hacia Estudiante. Evidencia un acoplamiento muy bajo. Significa que el sistema de reportes simplemente "usa" los datos del estudiante de forma transitoria pasándolo como parámetro en un método, sin almacenarlo como una variable permanente dentro de su propia clase.

Ejecución:

```
PS C:\Users\User\German_UNSA\GENAL> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetails -Djava.class.path=C:\Code\User\workspaceStorage\6f6de67c70998c1e49ed43356abc8123\redhat.java\jdt_ws\jdt.ls-java-project'
SISTEMA DE GESTIÓN UNIVERSITARIA

--- Profesores Registrados ---
Profesor [Persona [nombre=Dr. Carlos Ruiz, edad=48], especialidad=Sistemas Distribuidos]
Profesor [Persona [nombre=MSc. Ana Paredes, edad=39], especialidad=Arquitectura de Software]

--- Estudiantes Registrados ---
Estudiante [Persona [nombre=Juan Perez, edad=20], codigo=20220101]
Estudiante [Persona [nombre=Maria Lopez, edad=22], codigo=20210452]
Estudiante [Persona [nombre=Luis Quispe, edad=21], codigo=20221189]

--- Cursos Creados (con Horario por Composición) ---
Curso [nombre=Tecnologia de Objetos, Horario [dias=Lunes y Miercoles, horas=08:00 - 10:00]]
Curso [nombre=Bases de Datos Avanzadas, Horario [dias=Martes y Jueves, horas=14:00 - 16:00]]

--- Universidad y Cursos Agregados ---
Universidad [nombre=Universidad Nacional Tecnológica, cantidadCursos=2]
-> Curso [nombre=Tecnologia de Objetos, Horario [dias=Lunes y Miercoles, horas=08:00 - 10:00]]
-> Curso [nombre=Bases de Datos Avanzadas, Horario [dias=Martes y Jueves, horas=14:00 - 16:00]]

=====
REPORTE ACADÉMICO DE ESTUDIANTE
=====
Nombre: Juan Perez
Edad: 20 años
Código: 20220101
Estado: ACTIVO REGULAR
=====
PS C:\Users\User\German_UNSA\GENAL>
```

Versiones del code en Python y en C++

❖ C++

C/C++

```
#include <iostream>
#include <string>
#include <vector>
```

```
// 1. HERENCIA: Clase Base
// Usamos protected o private según corresponda, y getters/setters.
class Persona {
```

```
private:
    std::string nombre;
    int edad;

public:
    Persona() : nombre(""), edad(0) {}
    Persona(const std::string& nombre, int edad) : nombre(nombre), edad(edad) {}
    virtual ~Persona() = default;

    std::string getNombre() const { return nombre; }
    void setNombre(const std::string& n) { nombre = n; }

    int getEdad() const { return edad; }
    void setEdad(int e) { edad = e; }

    virtual std::string toString() const {
        return "Persona [nombre=" + nombre + ", edad=" + std::to_string(edad) + "];"
    }
};

// 1. HERENCIA: Clases Hijas
class Profesor : public Persona {
private:
    std::string especialidad;

public:
    Profesor() : Persona(), especialidad("") {}
    Profesor(const std::string& nombre, int edad, const std::string& especialidad)
        : Persona(nombre, edad), especialidad(especialidad) {}

    std::string getEspecialidad() const { return especialidad; }
    void setEspecialidad(const std::string& esp) { especialidad = esp; }

    std::string toString() const override {
        return "Profesor [" + Persona::toString() + ", especialidad=" + especialidad + "];"
    }
};

class Estudiante : public Persona {
private:
    std::string codigoEstudiante;

public:
    Estudiante() : Persona(), codigoEstudiante("") {}
    Estudiante(const std::string& nombre, int edad, const std::string& codigo)
        : Persona(nombre, edad), codigoEstudiante(codigo) {}

    std::string getCodigoEstudiante() const { return codigoEstudiante; }
    void setCodigoEstudiante(const std::string& cod) { codigoEstudiante = cod; }

    std::string toString() const override {
        return "Estudiante [" + Persona::toString() + ", codigo=" + codigoEstudiante + "];"
    }
}
```

```
};
```

```
// 2. COMPOSICIÓN: Horario es parte vital de Curso
```

```
class Horario {  
private:  
    std::string dias;  
    std::string horas;  
  
public:  
    Horario() : dias(""), horas("") {}  
    Horario(const std::string& dias, const std::string& horas) : dias(dias), horas(horas) {}  
  
    std::string getDias() const { return dias; }  
    void setDias(const std::string& d) { dias = d; }  
  
    std::string getHoras() const { return horas; }  
    void setHoras(const std::string& h) { horas = h; }  
  
    std::string toString() const {  
        return "Horario [dias=" + dias + ", horas=" + horas + "];"  
    }  
};
```

```
class Curso {  
private:  
    std::string nombre;  
    // Composición: Horario pertenece enteramente a Curso.  
    // En C++ esto se representa usando objetos como miembros por valor en vez de punteros.  
    Horario horario;  
  
public:  
    Curso(const std::string& nombre, const std::string& dias, const std::string& horas)  
        : nombre(nombre), horario(dias, horas) {} // Se instancia junto con el Curso  
  
    std::string getNombre() const { return nombre; }  
    void setNombre(const std::string& n) { nombre = n; }  
  
    Horario getHorario() const { return horario; }  
    void setHorario(const Horario& h) { horario = h; }  
  
    std::string toString() const {  
        return "Curso [nombre=" + nombre + ", " + horario.toString() + "];"  
    }  
};
```

```
// 3. AGREGACIÓN: Universidad contiene Cursos independientes
```

```
class Universidad {  
private:  
    std::string nombre;  
    // Agregación: La universidad conoce los cursos pero no es dueña absoluta de su ciclo de vida.  
    std::vector<Curso*> listaCursos;  
  
public:
```

```
Universidad() : nombre("") {}
Universidad(const std::string& nombre) : nombre(nombre) {}

std::string getNombre() const { return nombre; }
void setNombre(const std::string& n) { nombre = n; }

const std::vector<Curso*>& getListasCursos() const { return listaCursos; }

// Agregación: Se recibe un puntero al curso que existe fuera de este objeto.
void agregarCurso(Curso* curso) {
    listaCursos.push_back(curso);
}

std::string toString() const {
    return "Universidad [nombre=" + nombre + ", cantidadCursos=" +
std::to_string(listaCursos.size()) + "];"
}

};

// 4. DEPENDENCIA: Reporte usa temporalmente a Estudiante
class Reporte {
public:
    Reporte() = default;

    // Dependencia: Pasamos una referencia constante para evitar copias y marcar que solo la
    "usamos"
    void generarReporteEstudiante(const Estudiante& estudiante) const {
        std::cout << "=====\n";
        std::cout << "          REPORTE ACADÉMICO DE ESTUDIANTE          \n";
        std::cout << "=====\n";
        std::cout << "Nombre: " << estudiante.getNombre() << "\n";
        std::cout << "Edad:   " << estudiante.getEdad() << " años\n";
        std::cout << "Código: " << estudiante.getCodigoEstudiante() << "\n";
        std::cout << "Estado: ACTIVO REGULAR\n";
        std::cout << "=====\n\n";
    }

    std::string toString() const {
        return "Reporte [modulo=Generador de reportes temporales]";
    }
};

int main() {
    std::cout << "          SISTEMA DE GESTIÓN UNIVERSITARIA          \n\n\n";

    // 1. Creación de 2 profesores (Herencia)
    Profesor prof1("Dr. Carlos Ruiz", 48, "Sistemas Distribuidos");
    Profesor prof2("MSc. Ana Paredes", 39, "Arquitectura de Software");

    std::cout << "--- Profesores Registrados ---\n";
    std::cout << prof1.toString() << "\n";
    std::cout << prof2.toString() << "\n\n";

    // 2. Creación de 3 estudiantes (Herencia)
```

```
Estudiante est1("Juan Perez", 20, "20220101");
Estudiante est2("Maria Lopez", 22, "20210452");
Estudiante est3("Luis Quispe", 21, "20221189");

std::cout << "--- Estudiantes Registrados ---\n";
std::cout << est1.toString() << "\n";
std::cout << est2.toString() << "\n";
std::cout << est3.toString() << "\n\n";

// 3. Creación de 2 cursos (Composición con Horario)
Curso curso1("Tecnologia de Objetos", "Lunes y Miercoles", "08:00 - 10:00");
Curso curso2("Bases de Datos Avanzadas", "Martes y Jueves", "14:00 - 16:00");

std::cout << "--- Cursos Creados (con Horario por Composición) ---\n";
std::cout << curso1.toString() << "\n";
std::cout << curso2.toString() << "\n\n";

// 4. Agregar cursos a la Universidad (Agregación)
Universidad uni("Universidad Nacional Tecnológica");
uni.agregarCurso(&curso1); // Pasamos puntero del curso
uni.agregarCurso(&curso2);

std::cout << "--- Universidad y Cursos Agregados ---\n";
std::cout << uni.toString() << "\n";
for (const auto& c : uni.getListaCursos()) {
    std::cout << " -> " << c->toString() << "\n";
}
std::cout << "\n";

// 5. Generar reporte de un estudiante (Dependencia)
Reporte reporte;
reporte.generarReporteEstudiante(est1);

return 0;
}
```

❖ Python

Python

```
import typing
from typing import List
```

1. HERENCIA: Clase Base

En Python, todas las clases heredan implícitamente de `object`.

```
class Persona:
```

```
    def __init__(self, nombre: str = "", edad: int = 0) -> None:
        # En Python, el encapsulamiento se simula convencionalmente con un guion bajo
        # (para "protected") o dos (para "private"). Usamos getters y setters
        self._nombre: str = nombre
```

```
self._edad: int = edad

def get_nombre(self) -> str:
    return self._nombre

def set_nombre(self, nombre: str) -> None:
    self._nombre = nombre

def get_edad(self) -> int:
    return self._edad

def set_edad(self, edad: int) -> None:
    self._edad = edad

def __str__(self) -> str:
    return f"Persona [nombre={self._nombre}, edad={self._edad}]"
```

1. HERENCIA: Clases Hijas

En Python, la herencia se indica pasando la clase padre como parámetro en la definición de la clase.

```
class Profesor(Persona):
    def __init__(self, nombre: str = "", edad: int = 0, especialidad: str = "") -> None:
        # super().__init__() llama al constructor de la clase base, equivalente a super() en Java.
        super().__init__(nombre, edad)
        self._especialidad: str = especialidad

    def get_especialidad(self) -> str:
        return self._especialidad

    def set_especialidad(self, especialidad: str) -> None:
        self._especialidad = especialidad

    def __str__(self) -> str:
        # Llamar a super().__str__() es equivalente a super().toString() en Java.
        return f"Profesor [{super().__str__()}, especialidad={self._especialidad}]"

class Estudiante(Persona):
    def __init__(self, nombre: str = "", edad: int = 0, codigo_estudiante: str = "") -> None:
        super().__init__(nombre, edad)
        self._codigo_estudiante: str = codigo_estudiante

    def get_codigo_estudiante(self) -> str:
        return self._codigo_estudiante

    def set_codigo_estudiante(self, codigo_estudiante: str) -> None:
        self._codigo_estudiante = codigo_estudiante

    def __str__(self) -> str:
        return f"Estudiante [{super().__str__()}, codigo={self._codigo_estudiante}]"
```

2. COMPOSICIÓN: Horario es parte vital de Curso

```
class Horario:
    def __init__(self, dias: str = "", horas: str = "") -> None:
```

```
self._dias: str = dias
self._horas: str = horas

def get_dias(self) -> str:
    return self._dias

def set_dias(self, dias: str) -> None:
    self._dias = dias

def get_horas(self) -> str:
    return self._horas

def set_horas(self, horas: str) -> None:
    self._horas = horas

def __str__(self) -> str:
    return f"Horario [dias={self._dias}, horas={self._horas}]"

class Curso:
    def __init__(self, nombre: str, dias: str, horas: str) -> None:
        self._nombre: str = nombre
        # Composición: el Horario nace y está íntimamente ligado al Curso
        # Al igual que en Java, lo instanciamos internamente.
        self._horario: Horario = Horario(dias, horas)

    def get_nombre(self) -> str:
        return self._nombre

    def set_nombre(self, nombre: str) -> None:
        self._nombre = nombre

    def get_horario(self) -> Horario:
        return self._horario

    def set_horario(self, horario: Horario) -> None:
        self._horario = horario

    def __str__(self) -> str:
        return f"Curso [nombre={self._nombre}, {self._horario.__str__()}]"

# 3. AGREGACIÓN: Universidad contiene Cursos independientes
class Universidad:
    def __init__(self, nombre: str = "") -> None:
        self._nombre: str = nombre
        # Tipado para indicar que la lista contiene objetos de tipo Curso
        self._lista_cursos: List[Curso] = []

    def get_nombre(self) -> str:
        return self._nombre

    def set_nombre(self, nombre: str) -> None:
        self._nombre = nombre
```

```
def get_lista_cursos(self) -> List[Curso]:
    return self._lista_cursos

# Agregación: Recibe cursos ya existentes exteriormente
def agregar_curso(self, curso: Curso) -> None:
    self._lista_cursos.append(curso)

def __str__(self) -> str:
    return f"Universidad [nombre={self._nombre}, cantidadCursos={len(self._lista_cursos)}]"

# 4. DEPENDENCIA: Reporte usa temporalmente a Estudiante
class Reporte:
    def __init__(self) -> None:
        pass

# Dependencia: Estudiante pasa como parámetro temporal para la operación
def generar_reporte_estudiante(self, estudiante: Estudiante) -> None:
    print("=====")
    print("      REPORTE ACADÉMICO DE ESTUDIANTE      ")
    print("=====")
    print(f"Nombre: {estudiante.get_nombre()}")
    print(f"Edad:   {estudiante.get_edad()} años")
    print(f"Código: {estudiante.get_codigo_estudiante()}")
    print("Estado: ACTIVO REGULAR")
    print("=====\\n")

def __str__(self) -> str:
    return "Reporte [modulo=Generador de reportes temporales]"

# PROGRAMA PRINCIPAL (Main)
# Este bloque if __name__ == "__main__": simula el método public static void main(String[] args)
def main() -> None:
    print("      SISTEMA DE GESTIÓN UNIVERSITARIA      ")
    print("\\n")

    # 1. Creación de 2 profesores (Herencia)
    prof1: Profesor = Profesor("Dr. Carlos Ruiz", 48, "Sistemas Distribuidos")
    prof2: Profesor = Profesor("MSc. Ana Paredes", 39, "Arquitectura de Software")

    print("--- Profesores Registrados ---")
    print(prof1)
    print(prof2)
    print()

    # 2. Creación de 3 estudiantes (Herencia)
    est1: Estudiante = Estudiante("Juan Perez", 20, "20220101")
    est2: Estudiante = Estudiante("Maria Lopez", 22, "20210452")
    est3: Estudiante = Estudiante("Luis Quispe", 21, "20221189")

    print("--- Estudiantes Registrados ---")
    print(est1)
    print(est2)
    print(est3)
```

```
print()

# 3. Creación de 2 cursos (Composición con Horario)
curso1: Curso = Curso("Tecnología de Objetos", "Lunes y Miercoles", "08:00 - 10:00")
curso2: Curso = Curso("Bases de Datos Avanzadas", "Martes y Jueves", "14:00 - 16:00")

print("--- Cursos Creados (con Horario por Composición) ---")
print(curso1)
print(curso2)
print()

# 4. Agregar cursos a la Universidad (Agregación)
uni: Universidad = Universidad("Universidad Nacional Tecnológica")
uni.agregar_curso(curso1)
uni.agregar_curso(curso2)

print("--- Universidad y Cursos Agregados ---")
print(uni)
for c in uni.get_lista_cursos():
    print(f" -> {c}")
print()

# 5. Generar reporte de un estudiante (Dependencia)
reporte: Reporte = Reporte()
reporte.generar_reporte_estudiante(est1)

if __name__ == "__main__":
    main()
```

❖ Tabla Comparativa entre lenguajes

Aspecto	Java	Python	C++
Tiempos de ejecución y modelo	Balanceado; se compila a Bytecode y la JVM lo ejecuta, permitiendo su uso en cualquier SO.	Interpretado (más lento al ejecutar), ahorra tiempo al programar.	Compilado a código máquina; extremadamente rápido.
Sintaxis (facilidad de lectura/escritura)	Verboso y estricto, exige escribir bastante pero es predecible.	Muy amigable y fácil de escribir/leer, similar al inglés.	Intimidante; requiere control explícito de punteros y referencias.

Manejo de memoria

Automático con Garbage Collector.

Automático con Garbage Collector.

Manual (uso de new/delete, destructores o punteros inteligentes/RAII).

II. CUESTIONARIO**1. ¿Qué diferencias puede resaltar en las implementaciones de los dos lenguajes de programación en POO? (Comparativa: Java vs. Python)**

Al comparar la implementación de este modelo orientado a objetos entre **Java** y **Python**, destacan las siguientes diferencias fundamentales:

- **Mecanismo de Encapsulamiento y Visibilidad:**

- **Java:** Aplica un encapsulamiento estricto forzado por el compilador mediante palabras clave (private, protected, public). Si un miembro es declarado private, el acceso directo desde fuera de la clase produce un error de compilación en tiempo de diseño [3], [5].
- **Python:** La protección se basa en convenciones de nomenclatura (filosofía "*we are all consenting adults here*"). Un prefijo de un guion bajo (_atributo) denota uso interno por convención, mientras que un doble guion bajo (__atributo) activa el *name mangling* (alteración del nombre interno para evitar colisiones en herencia), pero no restringe físicamente el acceso en memoria en tiempo de ejecución [6].

- **Sistema de Tipado y Polimorfismo:**

- **Java:** Posee un tipado estático y nominal. Las relaciones de herencia y pasaje de parámetros (como en la clase Reporte que recibe Estudiante) exigen compatibilidad de tipos verificada en tiempo de compilación [3], [7].
- **Python:** Utiliza tipado dinámico y *duck typing* («si camina como pato y suena como pato, es un pato»). La clase Reporte puede recibir cualquier objeto siempre que exponga los métodos o atributos requeridos, sin necesidad de pertenecer formalmente a la jerarquía de Persona [6].

- **Verbosidad y Estructura Sintáctica:**

- **Java:** Requiere la implementación explícita del estándar JavaBeans (constructores vacíos, sobrecargados, pares de *getters* y *setters*, y métodos *toString()*), demandando mayor cantidad de líneas de código (*boilerplate*) [3].

- **Python:** Sintaxis concisa. La inicialización se centraliza en el método `__init__`, la representación textual en `__str__` o `__repr__`, y el acceso a datos suele hacerse directamente o mediante decoradores `@property` para emular *getters* y *setters* sin alterar la interfaz externa de la clase [6].

2. ¿En su implementación dónde se puede evidenciar la protección de datos o la seguridad utilizando la técnica de la programación orientada a objetos?

En la solución desarrollada en Java, la protección e integridad de datos se evidencia en tres puntos clave:

- **Ocultamiento de Información (*Information Hiding*) mediante Atributos Privados:**

- Todos los atributos de las entidades (nombre, edad, días, horas, listaCursos, etc.) fueron declarados explícitamente con el modificador `private` [2], [5]. Esto previene que clientes externos muten el estado interno del objeto de forma no autorizada o descontrolada (por ejemplo, impidiendo sentencias como `estudiante.edad = -5;`). El acceso se canaliza exclusivamente mediante métodos *getter* y *setter*, permitiendo introducir validaciones en el punto de entrada [3].

- **Control del Ciclo de Vida mediante Composición (Curso - Horario):**

- La relación de composición garantiza la protección estructural. La instancia de Horario se crea y vive encapsulada dentro del constructor de Curso. El objeto exterior no puede desligar el horario del curso ni crear un curso con un estado horario corrupto o nulo, protegiendo la invariante del modelo de dominio [4], [7].

- **Aislamiento de Estado en la Relación de Dependencia (Reporte - Estudiante):**

- La clase Reporte únicamente recibe la referencia de Estudiante dentro del ámbito local de ejecución del método `generarReporteEstudiante(Estudiante estudiante)`. La clase Reporte **no almacena ni retiene** la referencia del estudiante como un atributo de clase persistente. Esto evita retenciones indebidas de memoria (*memory leaks*) y minimiza el riesgo de que el estado del estudiante sea modificado colateralmente por el generador de reportes (*side effects*) [2].

III. CONCLUSIONES Y RETROALIMENTACIÓN GENERAL

Conclusiones del Laboratorio

- **Diferenciación Práctica de Relaciones Estructurales:** Se logró constatar que la diferencia entre agregación y composición radica en la propiedad del ciclo de vida de los objetos: mientras que Curso es dueño absoluto de la existencia de Horario (composición), Universidad simplemente agrupa cursos preexistentes cuya vida útil no depende de ella (agregación).
- **Eficiencia del Acoplamiento Débil en la Dependencia:** La implementación de la clase Reporte demostró las ventajas de la dependencia funcional sobre la asociación persistente, logrando que el componente generador sea reutilizable para cualquier estudiante sin consumir memoria residual tras su ejecución.
- **Robustez mediante Encapsulamiento:** La estandarización mediante JavaBeans y modificadores private demostró ser la primera línea de defensa en la seguridad del software a nivel de código fuente, asegurando que las transiciones de estado de los objetos solo ocurran bajo reglas predefinidas.
- **Interoperabilidad Conceptual:** El modelado formal de relaciones en POO es universal; sin embargo, lenguajes como Java garantizan estas restricciones mediante el compilador y tipos estáticos, mientras que alternativas dinámicas como Python delegan esta responsabilidad a la convención del desarrollador y pruebas automatizadas.

Retroalimentación General

- **Pertinencia Pedagógica:** El laboratorio cuenta con un diseño instruccional claro y progresivo. Integrar los cuatro tipos fundamentales de relación (herencia, agregación, composición y dependencia) dentro de un solo caso de estudio (entorno universitario) permite al estudiante contrastar visual y lógicamente la diferencia entre "ser un", "estar compuesto por", "agregar a" y "usar a".
- **Alineación con Estándares de la Industria:** Forzar la aplicación del patrón JavaBeans y la sobrescritura del método toString() fomenta buenos hábitos de desarrollo (código limpio, trazabilidad y facilidad para depuración mediante consolas o *loggers*).
- **Oportunidad de Mejora:** Sería provechoso para futuros laboratorios añadir reglas de validación en los *setters* (por ejemplo, validar que edad > 0 o que el formato de horas sea válido) para evidenciar de forma tangible la utilidad de no exponer campos públicos frente a errores en tiempo de ejecución.

IV. REFERENCIAS

- [1] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
- [3] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [4] B. Meyer, *Object-Oriented Software Construction*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
- [5] H. M. Deitel and P. J. Deitel, *Java How to Program, Early Objects*, 11th ed. Boston, MA, USA: Pearson, 2018.
- [6] L. Ramalho, *Fluent Python: Clear, Concise, and Effective Programming*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [7] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.